AI基础

01-数学基础

01-1 Linear Algebra

01-2 Regression

> **Question**
>
> 在一组数据点中找到一个多项式函数，拟合这组数据 (Linear/Polynomial Regression)
> 数据为 $\mathbf{x} = (x_1, \ldots, x_N)^T, \mathbf{t} = (t_1, \ldots, t_N)^T$
> 求"最佳"参数$\mathbf{w} = (w_0, \ldots, w_M)^T$

令M次多项式函数$y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j x^j$，误差函数$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(y(x_n, \mathbf{w}) - t_n)^2$，"最佳拟合"即为使得$E(\mathbb{w})$最小的$\mathbb{w}$

> **是不是$\mathbb{w}$越大越好呢?**
>
> 多项式次数过高会导致**过拟合**
> 训练数据过少，参数过多可能会**过拟合**

- 过拟合解决方案:
  - 增加数据量
  - 权重正则化

01-3 Probability Theory

CS229-Mathmatical Fundamentals

[CS229-Probability Review]

概率的定义

- 概率三要素
  - 样本空间Ω
  - 事件空间$\mathcal{F} \subset 2^{\Omega}$
  - 概率测度$P : \mathcal{F} \to [0, 1]$
- 联合概率:
  - $X = x_i$ 和 $Y = y_j$同时发生的概率

    - Sum Rule
        - $P(X=x_i)=\sum_{j=1}^{L}P(X=x_i,Y=y_j)$
- 条件概率
    - 在事件$X=x_i$发生的情况下，事件$Y=y_j$发生的概率
    - Product Rule
        - $P(X=x_i,Y=y_j)=P(Y=y_j|X=x_i)P(X=x_i)$

Beyes' theorem

$$P(Y|X)=\frac{P(X|Y)P(Y)}{\sum_Y P(X|Y)P(Y)}$$

- PDF与CDF
  - $f_X(x)=\frac{\mathrm{d}F_X(x)}{\mathrm{d}x}$
  - 所以$F_X(k)=\int_{-\infty}^{k}F_X(x)\mathrm{d}x$
  - 且
  $P(x\le X\le x+\Delta x)=F_X(x+\Delta x)-F_X(x)=\int_x^{x+\Delta x}f_X(x)\mathrm{d}x=f_X(\xi)\Delta x\approx f_X(x)\Delta x$
  感觉不如看CS229-Probability

02-机器学习-线性模型Part01

任务

- 有监督学习
    - 分类问题、回归问题…
- 无监督学习
    - 聚类、密度估计、降维…
- 半监督学习
- 弱监督学习
- 强化学习

经验

- 训练数据
- 测试数据

表现

- 表现该如何衡量?
  - 损失函数$loss=\mathcal{L}(f,\mathbb{D})$
  目标：构建预测器$f:\mathbb{X}\to\mathbb{Y}$，能够做出合理预测
  途径：最小化损失函数的期望$\mathbb{E}[\mathcal{L}(f,\mathbb{D})]$

线性回归

> **Note**
>
> 连续数值预测：回归问题
> 离散数值预测：分类问题

目标：构建一个预测器$f:\mathbb{X}\to\mathbb{Y}$来最小化损失函数
线性模型：$f(\mathbf{x})=\sum_{i=1}^{d}w_ix_i+\mathbf{b}=\mathbf{w}^\top\mathbf{x}+\mathbf{b}$
给定数据集：$\mathbb{D}=\{(\mathbf{x_1},y_1),(\mathbf{x_2},y_2),\cdots,(\mathbf{x_n},y_n)\}$，求$\mathbf{w}$使得

$$\mathcal{L}(\mathbf{w},\mathbb{D})=E[(f(\mathbf{x})-y)^2]$$

最小

由大数定理，我们使用 经验均值/Empirical Mean 估计期望：

$$\begin{aligned}l(\mathbf{w},\mathbb{D})&=\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}(f(\mathbf{x})-y)^2\\&=\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}\left(y-\mathbf{w}^\top\mathbf{x}-\mathbf{b}\right)^2\end{aligned}$$

从而

$$\begin{aligned}(\mathbf{w}^*,b^*)&=\arg\min_{(\mathbf{w},b)}\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}\left(y-\mathbf{w}^\top\mathbf{x}-\mathbf{b}\right)^2\\&=\frac{1}{|\mathbb{D}|}\arg\min_{\beta}\sum_{i=1}^{n}(\mathbf{X}_i\beta-y_i)^2\\&=\frac{1}{|\mathbb{D}|}\arg\min_{\beta}\|\mathbf{A}\beta-\mathbf{y}\|^2\end{aligned}$$

其中

$$\mathbf{X}_i^\top=\begin{pmatrix}x_i^{(1)}\\\vdots\\x_i^{(d)}\\1\end{pmatrix}\qquad\mathbf{A}=\begin{pmatrix}\mathbf{X}_1\\\vdots\\\mathbf{X}_n\end{pmatrix}\qquad\beta=\begin{pmatrix}w_1\\\vdots\\w_n\\b\end{pmatrix}\qquad\mathbf{y}=\begin{pmatrix}\mathbf{y}_1\\\vdots\\\mathbf{y}_n\end{pmatrix}$$

令函数$\mathcal{J}(\beta)=\|\mathbf{A}\beta-\mathbf{Y}\|^2=(\mathbf{A}\beta-\mathbf{Y})^\top(\mathbf{A}\beta-\mathbf{Y})=\beta^\top\mathbf{A}^\top\mathbf{A}\beta-\mathbf{Y}^\top\mathbf{A}\beta-\beta^\top\mathbf{A}^\top\mathbf{Y}+\mathbf{Y}^\top\mathbf{Y}$
求导：

$$\frac{\partial\mathcal{J}}{\partial\beta}|_{\hat\beta}=2\mathbf{A}^\top\mathbf{A}\hat\beta-2\mathbf{A}^\top\mathbf{y}=0$$

即

$$\mathbf{A}^\top\mathbf{A}\hat\beta=\mathbf{A}^\top\mathbf{y}$$

当$\mathbf{A}^\top\mathbf{A}$可逆时，可以计算

$$\widehat{\beta} = \left(\mathbf{A}^\top\mathbf{A}\right)^{-1}\mathbf{A}^\top\mathbf{y}$$

什么时候可逆？什么时候不可逆？

- 样本数量 < 特征维度时不可逆（方程组欠定）
  - 不可逆时使用正则化/Regularization

但是，当矩阵大小非常大时候，即使$\mathbf{A}^\top\mathbf{A}$可逆，求解矩阵的逆开销也是非常大的
有没有什么更好的方法？

> 考虑优化问题！

梯度下降法

求$J(\beta)$的最小值，由于它是convex function，可以通过计算梯度的方法迭代
Step1. Initialize: β_0
Step2. Update:

$$\beta_{t+1} = \beta_t - \frac{\alpha}{2}\frac{\partial \mathcal{J}}{\partial \beta}|_{\beta_t}$$

Note

Step-size α的作用：

- 这个数值的大小代表学习率/learning rate
- small α：收敛速度慢，但不容易在最值附近摆动
- large α：收敛速度快，但有可能摆动

Stop. Convergence Condition:

$$\frac{\partial \mathcal{J}}{\partial \beta}|_{\beta_t} < \epsilon$$

按照该算法计算，最终可以最小化损失函数，进而获得预测器f

但是，如果矩阵$\mathbf{A}^\top\mathbf{A}$不可逆，我们应该如何最小化损失函数呢？

考虑矩阵不可逆的情况，这说明了样本数量小于特征维度，也就是说，此时预测器获得的信息不足以构建出整个模型，考虑人为增加一些条件。即，通过假设的方式人为“增加样本的数量”

“假设”，在贝叶斯统计中，可以体现为prior distribution，从而，我们可以运用贝叶斯统计的理论来解决这个问题

贝叶斯统计(from 卓致用)

我们要构建这个预测器$f:\mathbb{X}\to\mathbb{Y}$，可以通过 最大后验估计/Maximum A Posterior 的方式
由于后验概率$p\left(\beta \mid D\right) \propto p(D|\beta)p(\beta)$，所以

$$\max_{\beta} p(D|\beta)p(\beta) = \max_{\beta} \log p(D|\beta) + \log p(\beta)$$

$$\hat{\beta}_{\text{MAP}} = arg\max_{\beta} \underbrace{\log p\left(\{(X_i, Y_i)\}_{i=1}^{n} \mid \beta, \sigma^2\right)}_{\text{log likelihood}} + \underbrace{logp\left(\beta\right)}_{\text{log prior}}$$

第一行的含义是，我们要最大化数据的似然函数和参数的先验概率的乘积。

- 似然函数表示数据（D）在给定参数 β 的情况下出现的概率
- 先验概率表示我们在看到数据之前对参数的信念。

最大化这个乘积等价于最大化它们的对数，因为对数是单调递增的函数。

第二行的第一个项代表的含义是，在给定参数 β 和方差 σ^2 的情况下，数据出现的概率。

> 在统计模型中：
>
> - β 通常代表模型的系数
> - σ^2 代表模型中的噪声或误差的方差。即使我们有了 β，我们还需要知道数据中的变异性（或不确定性）有多大，这就是为什么 σ^2 是重要的。
>
> 在某些模型，比如线性回归中，我们假设数据 Y 是由自变量 X 的线性组合（由 β 确定）加上一些随机噪声（由 σ^2 描述）生成的。这个噪声代表了除了 X 影响 Y 之外的其它因素。所以，σ^2 帮助我们了解除了主要效应（由 β 描述）之外，数据中还有多少随机波动。

我们采取高斯先验，也即假设参数遵循高斯分布（也称为正态分布）。那么也就有
$\beta \sim N(0, \tau^2 I)$。

这表示：

- 参数 β 是一个 N 维向量，其中 N 是特征的数量
- 其遵循均值为 0，方差为 τ^2 的多元高斯分布
- $\tau^2 I$ 是指协方差矩阵为 τ^2 的对角矩阵，这代表 β 的每个元素都是相互独立的，且每个元素的方差都是 τ^2。
- 也就是说，各个维度独立同分布（iid）。

β 的概率密度分布函数按照矩阵表示，则是：

$$p(\beta) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(\beta-\mu)^T\Sigma^{-1}(\beta-\mu)}$$

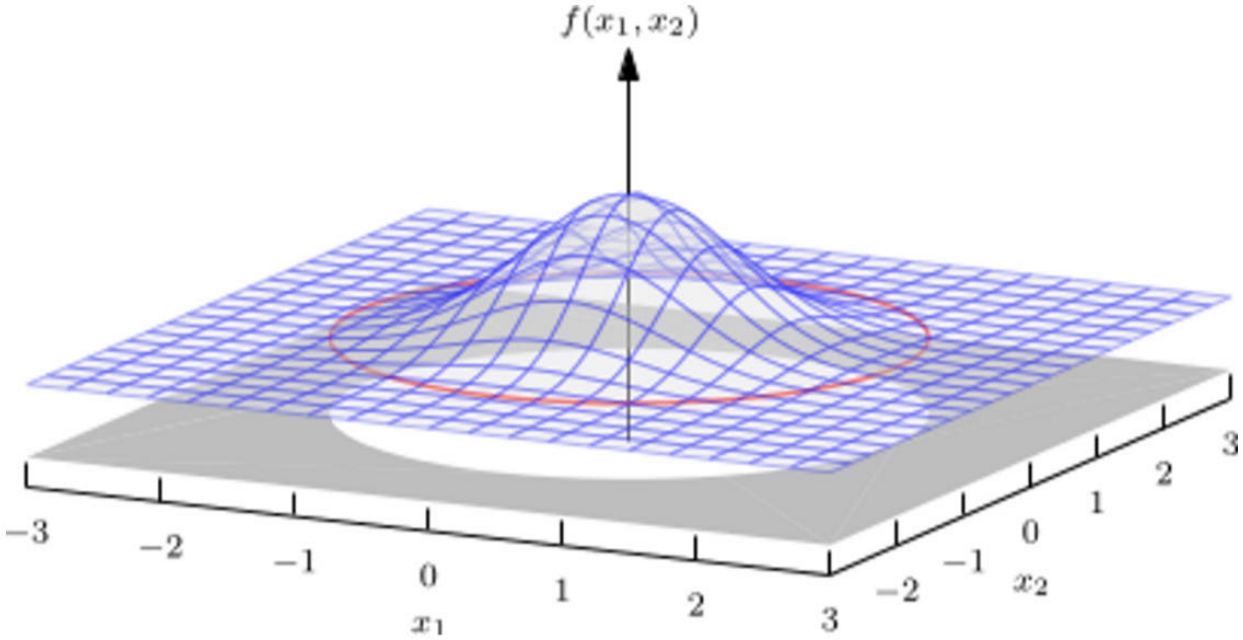
> 可以比较一维版本来理解~
>
> 其中的 $|\Sigma|$ 是协方差矩阵的行列式，它的作用是保证概率密度函数的总面积为 1。而当它为对角矩阵时，还能额外保证各个维度彼此独立。

所以，我们有 $p(\beta) \propto e^{-\beta^T\beta/2\tau^2}$。

这不包括归一化常数。$\propto$ 表示成比例，意味着这是未归一化的概率密度，它的形状随着 β 的变化而变化，但是总面积（概率总和）是固定的。$\beta^T\beta$ 是 β 的二次项，表示参数向量的长度的平方。

关于这一部分，可以阅读 [钱默吟/多元高斯分布完全解析]，讲的很详细。

这个式子的一个直观理解，就是假设 β 是一个二维的，那么我们可以可视化这个分布：



那么在这个图中，任意一个点都代表一个高斯分布，而 $f(x_1, x_2)$ 就是这个点的概率密度。我们可以看到，这个分布是关于原点对称的，这也是 $p(\beta)$ 中的 $\beta^T\beta$ 的作用，这代表了我们对于变量的顺序是没有偏好的。

由于$p(\beta) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(\beta-\mu)^{T\Sigma^{-1}(\beta-\mu)}} \propto e^{-\frac{1}{2}(\beta-\mu)^T\Sigma^{-1}(\beta-\mu)}$ 所以，$\log p(\beta) \propto \| \beta \|_2^2$

$$\hat{\beta}_{\mathbf{MAP}} = arg\min \sum_{i=1}^{n} (Y_i - X_i\beta)^2 + \lambda \| \beta \|_2^2$$

为什么刚才是 $arg\max$，现在又变成了 $arg\min$？简单说明就是，刚才我们要最大化从已知的 β 中得到数据真实分布的概率，这就等价于最小化使用我们的模型，从 X 中得到的数据的误差（也即最大似然估计）。

而 λ 是一个超参数，它的作用是控制我们对于参数的偏好，也就是我们对于模型的复杂度的偏好。在后续的学习中，我们会知道 $\lambda \| \beta \|_2^2$ 其实是一个正则项，它的作用是防止过拟合。

当我们对于 β 的先验假设不一样时，这个惩罚项（正则项）也会不一样。但大致原理不变，于是我们对这个形式加以推广，得到了类似如下的公式：

$$min_\beta (\mathbf{A}\beta - \mathbf{Y})^T(\mathbf{A}\beta - \mathbf{Y}) + \lambda \text{pen}(\beta) = minJ(\beta) + \lambda \text{pen}(\beta)$$

其中，$J(\beta)$ 是我们的损失函数，$\text{pen}(\beta)$ 是惩罚项。

对于惩罚项，我们一般有两个选择：

- L1 正则（1-范数）：$\text{pen}(\beta) = \| \beta \|_1 = \sum_{i=1}^{n} |\beta_i|$
- L2 正则（2-范数）：$\text{pen}(\beta) = \| \beta \|_2^2 = \sum_{i=1}^{n} \beta_i^2$

最终，我们得到**广义的线性模型**，我们可以考虑任意一种单调可微的函数 $g(\cdot)$：

$$y = g^{-1}(w^T x + b)$$

其中，$g(\cdot)$ 是激活函数，它的作用是将线性模型的输出转换为我们想要的输出，也即将线性模型的输出转换为一个非线性的预测值。比如，当 $g(\cdot)$ 是恒等函数时，我们就得到了线性回归；当 $g(\cdot)$ 是 sigmoid 函数时，我们就得到了逻辑回归。

02-机器学习-线性模型Part02

- Logistic Regression/Classification

Question

如何用线性模型将X分类到离散labels Y?

- 找一个单调可微的函数将Y和预测值联系起来
- **Unit-step function.**

$$y(z) = \begin{cases} 0, & z < 0 \\ 0.5, & z = 0 \\ 1, & z > 0 \end{cases}$$

- **Sigmoid (Logistic) function.**

$$y(z) = \frac{1}{1 + e^{-z}}$$

其中$z = w^T x + b$，从而：

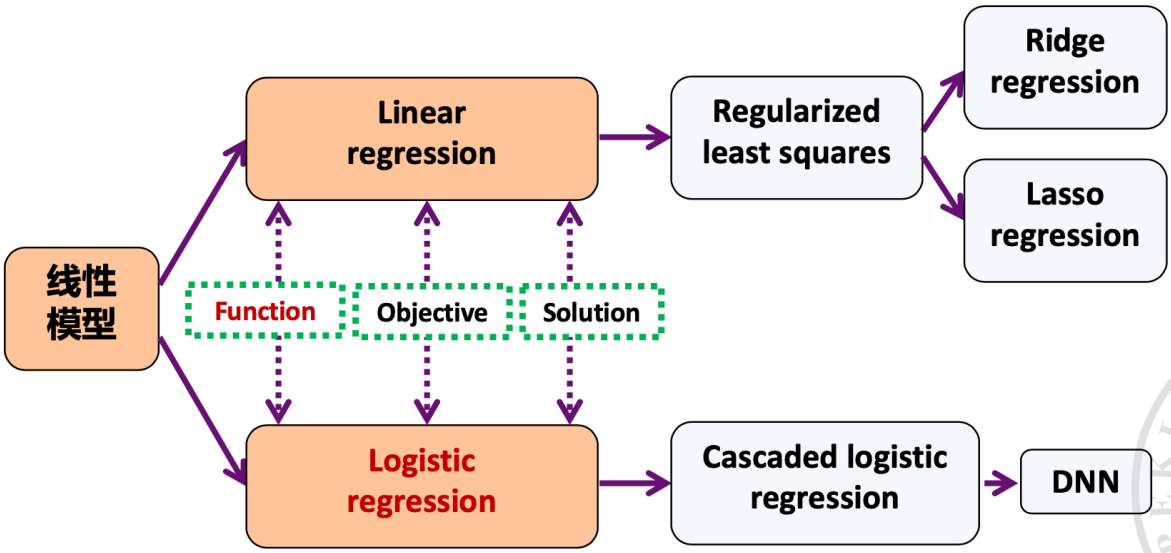
$$y = \frac{1}{1 + e^{-(w^T x + b)}}$$

也可以表示为：

$$\log \frac{y}{1 - y} = w^T x + b$$

Note

$$\frac{\partial sigmoid(x)}{\partial x} = sigmoid(x)\,(1 - sigmoid(x))$$



训练步骤：

Step1. 给定$\mathbf{x}$估计其出现$C_1 \mid x$的概率$P_{w,b}(C_1 \mid \mathbf{x}) = f_{w,b}(\mathbf{x})$，其中$f_{w,b}(\mathbf{x}) = \sigma\left(\sum_i w_i x_i + b\right)$

Step2. 找出目标函数/Objective function
给定训练数据

$$\begin{array}{ccccc} x^1 & x^2 & x^3 & \cdots\cdots & x^N \\ C_1 & C_1 & C_2 & \cdots\cdots & C_1 \end{array}$$

假设这些数据是基于$P_{w,b}(C_1 \mid \mathbf{x}) = f_{w,b}(\mathbf{x})$生成的
那么给定$\mathbf{w}$与$\mathbf{b}$，生成这些数据的概率是：

$$\mathcal{L}(w,b) = f_{w,b_r}(x^1) f_{w,b}(x^2)\left(1 - f_{w,b}(x^3)\right)\cdots f_{w,b}(x^N)$$

所以最优的$\mathbf{w}^*, \mathbf{b}^*$是：

$$w^*, b^* = \arg\max_{w,b} \mathcal{L}(w,b)$$

记label $\hat{y}$ 满足

$$\hat{y} = \begin{cases} 1, & C_1 \\ 0, & C_2 \end{cases}$$

所以

$$w^*, b^* = arg\max_{w,b} \mathcal{L}(w,b) = arg\min_{w,b} -log\mathcal{L}(w,b)$$

从而

$$\begin{aligned} -log\mathcal{L}(w,b) &= -\left(log f_{w,b}(x^1) + log f_{w,b}(x^2) + log\left(1 - f_{w,b}(x^3)\right)\cdots + log f_{w,b}(x^N)\right. \\ &= \underbrace{\sum_n -\left[\hat{y}^n log f_{w,b}(x^n) + (1-\hat{y}^n) log\left(1 - f_{w,b}(x^n)\right)\right]}_{\text{Cross Entropy}} \end{aligned}$$

Important

上面目标函数的负对数恰好是交叉熵/Cross Entropy

- 假设现在一个样本集合中有两个概率分布p与q，其中p代表真实分布，q代表非真实分布
- 用p衡量一个样本所需要的编码长度的期望为Cross Entropy：

$$H(p) = \sum_i p(i) \log(\frac{1}{q(i)})$$

- 对于离散变量：

$$H(p) = \sum_i p(i) \log(\frac{1}{q(i)})$$

- 对于连续变量：

$$H(p) = -\int_X P(x) \log Q(x) \mathrm{d}x$$

Step3. 梯度下降
计算梯度

$$\begin{aligned}
\frac{\partial(-log\mathcal{L}(w,b))}{\partial w_i} &= \sum_n -\left[\hat{y}^n \frac{\partial}{\partial w_i} log f_{w,b}(x^n) + (1-\hat{y}^n)\frac{\partial}{\partial w_i} log\left(1 - f_{w,b}(x^n)\right)\right] \\
&= \sum_n -\left[\hat{y}^n(1-f_{w,b}(x^n))x_i^n + (1-\hat{y}^n)f_{w,b}(x^n)x_i^n\right] \\
&= \sum_n -\left(\hat{y}^n - f_{w,b}(x^n)\right)x_i^n \\
&\propto \text{difference between label and prediction}
\end{aligned}$$

更新weight

$$w_i \leftarrow w_i - \eta \sum_n -\left(\hat{y}^n - f_{w,b}(x^n)\right)x_i^n$$

Question

为什么不用Square Error Loss Function?

如果用Square Error：

$$\mathcal{L}(f) = \frac{1}{2}\sum_n \left(f_{w,b}(x^n) - \hat{y}^n\right)^2$$

求梯度：

$$\frac{\partial \mathcal{L}}{\partial w_i} = (f_{w,b}(x) - \hat{y}) f_{w,b}(x) \left(1 - f_{w,b}(x)\right) x_i$$

无论$f_{w,b}(x^i) = 1/0$ 梯度都是0

> Logistic Regression局限性：
>
> - 无法处理多分类问题
> - 无法处理非线性问题

基于线性分类器的图片分类

图片分类：Computer Vision中的一个重要问题

- 问题：给定一张图片，判断它属于哪个类别
- 输入：图片
  - 一张图片可以看作是一个像素tensor
    - 例如 $32 \times 32 \times 3$ (RGB 3 channels)
- 输出：类别
- 任务：分类问题
- 挑战：
  - 图片的角度不一
  - 图片的光照不一
  - 图片的背景不一
  - 图片的遮挡不一
  - 图片的噪声不一

我们想要设计一个hard-code的算法来解决这个问题，但是这个问题太复杂了，我们无法hard-code所有的规则，所以我们需要使用data-driven的方法来解决这个问题，也就是机器学习的方法。

- 机器学习：数据驱动的方法
  - 收集图片和标签的数据集
  - 使用机器学习算法训练一个分类器
  - 在测试图片上评估此分类器

线性分类器/Linear Classifier

线性分类器是一个典型有参数的分类器，它的输出是输入的线性组合，然后通过一个阈值函数来判断输出的类别。

例如，对于一个$32 \times 32 \times 3$的图片，我们可以将其展开为一个$32 \times 32 \times 3 = 3072$维的行向量，然后我们可以找到一个最优的参数$W$和$b$，使得$xW + b$能够最好的区分不同的类别。

如果我们给定10个标签，将$32 \times 32 \times 3$的图片分类到这10个labels中，那么我们可以得到一个3072×10的矩阵W和一个10×1的向量b。

预测器：$f: \mathbb{R}^{3072 \times 1} \to \mathbb{R}^{10 \times 1}$满足$f(x, W) = xW + b$

那么，我们该如何找到这个Linear Classifier的最优参数W^*与b^*呢？

- 定义一个可以衡量训练数据上预测标签和真实标签差异的loss function
- 给定train dataset $\{(x_i, y_i)\}_i^N$
- 求解

$$W^*, b^* = \arg\min_{W,b} \sum_{i=1}^{N} \mathcal{L}(f(x_i, W), y_i)$$

多项式逻辑回归分类器/Multinomial Logistic Regression

我们从原始的线性分类器$f(x, W) = xW + b$得到一个"score vector"，但是这个score vector并不是一个概率分布，我们需要将其转换为一个概率分布。

使用Softmax函数：

$$P(Y = k \mid X = x_i) = \frac{\exp(s_k)}{\sum_j \exp(s_j)}$$

注意：向量中的分量有可能很大，导致指数爆炸问题，所以我们需要预处理向量
$x_i \leftarrow x_i - \max_j \left\{x_i^j\right\}$

由于

$$\frac{\exp(x_i^k - \max_j \left\{x_i^j\right\})}{\sum_k \exp(x_i^k - \max_j \left\{x_i^j\right\})} = \frac{\exp(s_k)}{\sum_j \exp(s_j)}$$

所以这样做是合理的

求得概率分布y_{pred}与真实标签y_{true}后，我们需要找到合适的Loss Function来衡量差异：

在这里，我们选择Cross Entropy Loss Function：

$$\mathcal{L}(y_{\text{pred}}, y_{\text{true}}) = -\sum_k y_{\text{true}}^k \log y_{\text{pred}}^k$$

最近邻分类器/Nearest Neighbor Classifier

K-近邻分类器是一种无参的方法，但是仍然是有监督学习，有训练数据

训练数据：$\{(x_i, y_i)\}_{i=1}^N$

预测器：$f: \mathbb{R}^{3072 \times 1} \to \mathbb{R}^{10 \times 1}$

训练：存储训练数据

预测：对于一个新的输入x，找到训练数据中与x最相似的k个数据点，然后返回这k个数据点中出现最多的标签

那么，什么叫做“最相似”呢，我们需要定义一个距离函数来衡量两个数据点之间的相似度，通常我们使用以下几种Distance Metric：

- L_1距离：

$$d(x_i, x_j) = \sum_k |x_i^k - x_j^k|$$

- L_2距离：

$$d(x_i, x_j) = \sqrt{\sum_k (x_i^k - x_j^k)^2}$$

其中，k与Distance Metric依赖于我们的选择，这种需要自行选择的参数称为 超参数/Hyperparameter

- 参数：在训练过程中需要学习的参数
  - 例如：W与b
- 超参数：在训练之前需要选择的参数
  - 例如：k与Distance Metric
  - 依赖于数据集与任务

聚类/Clustering

聚类是一种无监督学习，没有标签的数据集，目标是找到数据集中的一些内在的结构，把相似的数据聚到一起

聚类的特点是：

- 高类内相似度
- 低类间相似度

我们如果想要将数据集中的数据聚到一起，我们需要定义一个相似度/距离的度量，然后我们可以使用K-Means算法来实现这个目标

Distance Metric:

- L_1距离

$$d(x, y) = \sum_k |x^k - y^k|$$

- L_2距离

$$d(x,y)=\sqrt{\sum_k (x^k-y^k)^2}$$

- L_∞距离

$$d(x,y)=\max_k |x^k-y^k|$$

聚类算法：

- 分割方法/Partition algorithm
  - K-Means
  - Mixture-Model based clustering
- 层级方法/Hierarchical algorithm
  - Single-linkage
  - Average-linkage
  - Complete-linkage
  - Centroid-based

分割算法：把n个对象分割成K个簇（Clusters），使得每个对象属于且仅属于一个组。

目标：最优化某个选定分割标准的K组分割。

K-means是一种常用的聚类算法，旨在将数据集分成K个簇，使得同一个簇内的数据点之间距离尽可能小，而不同簇之间的数据点距离尽可能大。算法步骤如下：

算法步骤：

1. 随机选择K个数据点作为初始簇中心。(随机选择的K个数据点即为“种子”)
2. 将每个数据点分配给最近的簇中心，形成K个簇。
3. 重新计算每个簇的中心，即每个簇内所有数据点的均值。
4. 重复步骤 2 和 3，直到簇中心不再变化或变化非常小。

其他问题：

1. 对于种子的选择敏感，需要尝试不同的初始种子，且尽量让k个初始种子互相远离。
2. 受离群值（Outliers）影响较大。

神经网络基础

单个神经元

对于单个神经元来说，它有1个输出z与n个输入$\{x_i\}_{i\geq 1}^n$，输出是输入的 线性组合/linear combination

$$z = \sum_{i=1}^{n} w_i x_i$$

- 一个神经元是只有一个输出的单层神经网络
- 因为输出与所有输入连接，这层神经网络可以被称为 全连接层/full connected layer or dense layer

对于神经元的计算，我们可以使用 向量/vector 来表示，通常有列格式与行格式两种表示方法

列格式（常见于书籍）：
$w = (w_1 \quad w_2 \quad \cdots \quad w_n)^\top, x = (x_1 \quad x_2 \quad \cdots \quad x_n)^\top, z = w^\top x$
行格式（常见于代码）：
$w = (w_1 \quad w_2 \quad \cdots \quad w_n)^\top, x = (x_1 \quad x_2 \quad \cdots \quad x_n), z = xw$

偏置/bias：$b \in \mathbb{R}$使得输出值可以上下浮动，以更好的适应输入值

$$z = \sum_{i=1}^{n} w_i x_i + b$$

```
import torch
# Single neuro

# matrix multiplication 矩阵乘法
x = torch.tensor([[1., 2., 3.]])          # 1x3
w = torch.tensor([[-0.5], [0.2], [0.1]]) # 3x1
print("X:\n", x, "\nShape:", x.shape)
print("W:\n", w, "\nShape", w.shape)

# Bias 偏置
b1 = torch.tensor(0.5)

# 计算有偏置的矩阵乘法
z1 = torch.matmul(x, w) + b1
print("Z:\n", z1, "\nShape", z1.shape)
```

对于分类问题，偏置可以改变决策平面的位置，从而使得分类器更加灵活

- 两个输入特征:决策边界是一条线
- 三个输入特征:决策边界是一条平面
- 更多个输入特征:决策边界是一条 超平面/hyperplane

若两个神经元连接在一个（3 inputs - 2 outputs）

$$(z_1 \quad z_2) = (x_1 \quad x_2 \quad x_3) \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{pmatrix} + (b_1 \quad b_2)$$

```
# Two outputs 两个输出
x = torch.tensor([[1., 2., 3.]]) # 1x3
w = torch.tensor([[-0.5 ,  -0.3],
                                  [0.2  ,  0.4],
                              [0.1  ,  0.15]]) # 3x2
print("X:\n", x, "\nShape:", x.shape)
print("W:\n", w, "\nShape", w.shape)

# Bias 偏置
b2 = torch.tensor([0.5, 0.4])
z2 = torch.matmul(x, w) + b2
print("Z:\n", z2, "\nShape", z2.shape)
```

激活函数

激活函数是神经网络中的一个重要组成部分，它的作用是引入非线性性，使得神经网络可以拟合更加复杂的函数

sigmoid函数

给定一个神经网络，输出可以表示分数，比如踢足球的概率。为了表达0的概率，需要把输出值限制到0 1之间。

$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

```
z1 = torch.Tensor([[0.7000]]) # 线性组合值
# Activation function 激活函数
# Sigmoid function
sigmoid_torch = torch.nn.Sigmoid()
a1 = sigmoid_torch(z1)
print("Result torch sigmoid:", a1)

# define your own activation function
class ActSigmoid(torch.nn.Module):
    def __init__(self):
        super(ActSigmoid, self).__init__()
    def forward(self, x):
        return 1/(1+torch.exp(-x))

sigmoid_act = ActSigmoid()
a1_m = sigmoid_act(z1)
print("Result your own sigmoid:", a1_m)
```

tanh函数

与Sigmoid函数类似，Hyperbolic tangent(tanh)函数也是限制输出值到一个范围。但Tanh函数限制的范围是 -1~1，该函数往往用于回归任务(regression task)，比如网络输出图像，而图像的像素数值在 -1~1之间;再比如输出情绪，-1代表不开心，1代表开心，0代表中间状态。

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 1 - \frac{2}{1 + e^{2z}}$$

ReLU函数

ReLU函数把负值输出设为0，正值输出不变。可用作特征选取(feature selection)和简化网络优化(optimisation)计算(后文会讲解)

$$f(z) = \max(0, z) = \begin{cases} 0, \text{if } z \leq 0 \\ z, \text{if } z > 0 \end{cases}$$

Leaky ReLU函数

然而把负输出设为0会丢失输入信息导致，一个解决方案是采用Leaky ReLU函数。通过一个很小 是正数α(如0.1、0.2)来控制斜率(slop)，使得负输出的信息可以在激活值上体现。此外，Parametric ReLU(有参数的ReLU，也称PReLU)将α当作一个网络的参数，在后续优化过程中自行学习调整斜率。

$$f(z) = \begin{cases} \alpha z, \text{if } z \leq 0 \\ z, \text{if } z > 0 \end{cases}$$

Softmax函数

一个网络有多个输出的时候，我们可以输出多个情况下的概率，实现多分类任务 (multi-class classification)，例如输出选择踢足球、打篮球、跑步的概率。Softmax 是用在多分类任务下网络输出层上的函数，用以让所有激活值表示0~100%的概率， 并让所有激活值之和为1，则100%。

给定一个输入向量$z = (z_1 \quad z_2 \quad \cdots \quad z_n)$，向量维度是$n$，输出向量

$$softmax(z)^k = \frac{e^{z_k}}{\sum_{i=1}^{n} e^{z_i}}$$

```
# Softmax
z2 = torch.Tensor([[0.7000, 1.3500]])
softmax_torch = torch.nn.Softmax(dim=1)
a2 = softmax_torch(z2) # z2是神经网络未激活的值
print("Result torch softmax:", a2)

# define your own activation function
class ActSoftmax(torch.nn.Module):
        def __init__(self):
                super(ActSoftmax, self).__init__()
        def forward(self, x):
                return torch.exp(x)/torch.sum(torch.exp(x), dim=1, keepdim=True)
```

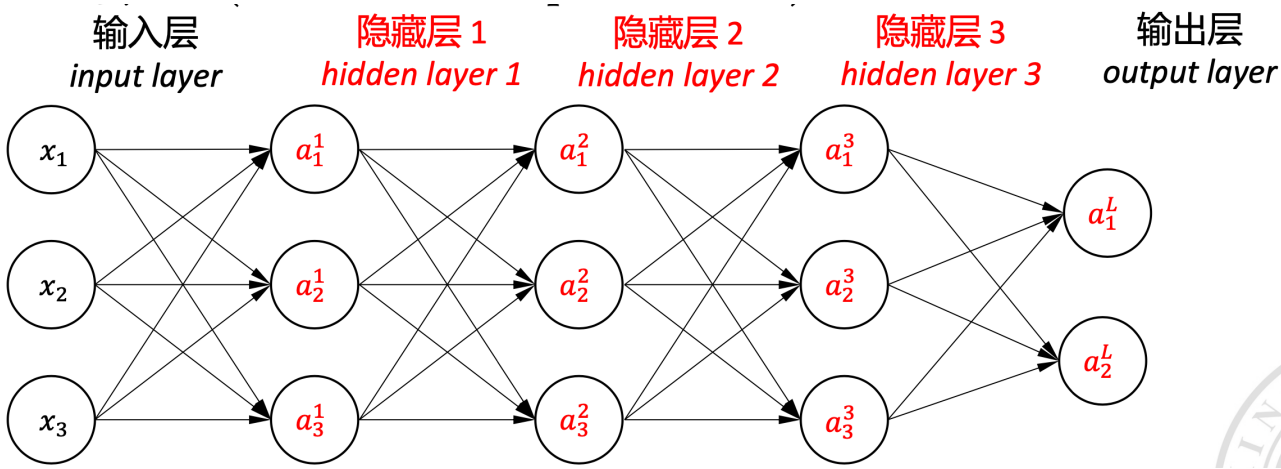
```
softmax_act = ActSoftmax()
a2_m = softmax_act(z2)
print("Result your own softmax:", a2_m)
```

多层感知器

多层感知器(multi-layer perceptron，MLP)在单层全连接网络上拓展，至少有2层全连接层。

在原有层上堆叠新的层，可以被认为是将原有层的输出值当作特征值来学习

因此，和单层全连接网络相比，MLP可以处理更复杂的输入数据，具有更好的 表达能力/representation capacity



损失函数

目的：

1. 损失函数/loss function 用来量化网络 预测的输出/predicted output 和 给定训练数据输出/ground truth 之间的 误差/error or loss value
2. 损失函数用来设定优化神经网络参数(如权重、偏置)的目标
3. 优化神经网络的过程，是通过更新其参数来使得误差尽可能小
4. 梯度下降/gradient descent 是最常用的优化方法，后文将有介绍

逻辑回归损失/Logistic Regression Loss

二值分类：

$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

M个数据样本：

$$\mathcal{L} = \sum_{m=1}^{M}(y^m\, log(a^m) + (1-y^m)\log(1-a^m))$$

交叉熵损失函数/Cross-Entropy Loss

M个数据样本：

$$\mathcal{L} = \sum_{m=1}^{M} y_k^m \log(a_k^m)$$

```
output = mlp(x_input)# 前向传播

# 多分类交叉熵
target = torch.tensor([[0, 1, 0]],dtype=torch.float) # 1x3
print("Target value:{}".format(target.numpy()))
print("Neural network output:{}".format(output.detach().numpy()))
loss_fn = torch.nn.CrossEntropyLoss()
l_ce = loss_fn(output, target)
print("Loss cross entropy:", l_ce)
```

L_p范数/norm

$$\|x\|_p^p = \sum_{k=1}^{K} |x_k|^p$$

均方误差/Mean Squared Error

$$\mathcal{L}_{MSE} = \frac{1}{M} \sum_{m=1}^{M} \|\mathbf{y}^m - \mathbf{a}^m\|_2^2$$

平均绝对误差/Mean Absolute Error

$$\mathcal{L}_{MAE} = \frac{1}{M} \sum_{m=1}^{M} |y^m - a^m|$$

```
# Mean absolute error 平均绝对误差
def mae(output, target):
        return torch.mean(torch.abs(output - target))
# Mean squared error 平均平方误差
def mse(output, target):
        return torch.mean((output - target)**2) y1 = torch.tensor([[1., 3., 5.,
7.]])
y2 = torch.tensor([[2., 4., 6., 8.]])
# 计算MAE和MSE
l_mae = mae(y1, y2)
l_mse = mse(y1, y2)
print("Loss MAE: {} \nLoss MSE: {}".format(l_mae, l_mse))
mse_torch = torch.nn.MSELoss()
l_mse_torch = mse_torch(y1, y2)
print("Loss MSE torch: {}".format(l_mse_torch))
```

优化

目的:

给定网络结构$f(x;\theta)$ 、定义好的损失函数$\mathcal{L}$和训练数据样本，训练网络的过程是通过更新网络参数θ(比如权重、偏置)以最小化误差的过程。

梯度下降(Gradient descent)是一个更新网络参数的方法，此外依然有很多优化方法存在，比如BFGS (L-BFGS) 和 conjugate gradient (CG)，但这些方法往往需要很多计算量，因此很少使用。

梯度下降法/Gradient Descent

$$\theta := \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

其中α是学习率/learning rate，是一个超参数，需要手动调整

误差反向传播/Back propagation

误差反向传播是用来计算网络中所有参数的梯度$\frac{\partial \mathcal{L}}{\partial \theta}$的方法。计算梯度时，通过引入中间结果$\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$，利用 链式法则/Chain Rule 可以逐层向前（反向！）计算梯度

算法（行格式）

Notation. $\circ$为Hadamard product，逐点相乘

1. 已知:

$$\begin{aligned}
\mathbf{a}^l &= f(\mathbf{z}^l) = \frac{1}{1+e^{-\mathbf{z}^l}} \\
\mathbf{z}^l &= \mathbf{a}^{l-1}\mathbf{W}^l + \mathbf{b}^l \\
\mathcal{L} &= \frac{1}{2}(\mathbf{y} - \mathbf{a}^l)^2
\end{aligned}$$

2. 求导:

\begin{align*} \frac{\partial \mathbf{a}^{l}}{\partial \mathbf{z}^{l}} &= f'(\math

\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{l}}&= \mathbf{a}^{l} - \mathbf{y}\\

\frac{\partial z^{l}}{\partial \mathbf{W}^{l}}&= \mathbf{a}^{l-1}~\mathrm{and}~\frac{\partial z^{l}}{\partial b^{l}}=1
\end{align*}

3.输出层的中间结果:

\delta^{l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{l}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{l}}\frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^{l}}=(\mathbf{a}^{l} -

\mathbf{y})\circ(\mathbf{a}^{l}\circ(1-\mathbf{a}^l))$$
4. 其他层的中间结果

$$\delta^l = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{l+1}} \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l} = \delta^{l+1} \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l}$$

$$\mathbf{z}^{l+1} = \mathbf{a}^l \mathbf{W}^{l+1} + \mathbf{b}^{l+1}$$

$$\frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l} = \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{a}^l} \frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^l} = \mathbf{W}^{l+1} \circ (\mathbf{a}^l \circ (1 - \mathbf{a}^l))$$

$$\delta^l = \delta^{l+1} \mathbf{W}^{l+1} \circ (\mathbf{a}^l \circ (1 - \mathbf{a}^l))$$

5. 计算梯度

\begin{align*} \frac{\partial\mathcal{L}}{\partial \mathbf{W}^{l}}&= \frac{\partia

\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{l}} &= \frac{\partial\mathcal{L}}{\partial \mathbf{z}^{l}}\frac{\partial \mathbf{z}^{l}}{\partial \mathbf{b}^{l}} =\delta^l
\end{align*}$$

6. 更新参数

\begin{align*} \mathbf{W}^{l}&:= \mathbf{W}^{l} - \alpha\frac{\partial\mathcal{L}}{

\mathbf{b}^{l}&:= \mathbf{b}^{l} - \alpha\frac{\partial\mathcal{L}}{\partial \mathbf{b}^{l}}
\end{align*}$$

梯度消失问题

刚刚的例子中，中间结果δ中有一项$(\mathbf{a}^l \circ (1 - \mathbf{a}^l))$，当激活输出$\mathbf{a}^l$接近0或者1时，中间结果$\delta$会变得很小。由于$\delta^l$又跟$\delta^{l+1}$有关，当反向传播时\delta比较小的话，会使得$\delta$越传播越小，使得靠近输入层的参数无法被更新，影响网络训练。

解决方法：

1. 用ReLU代替Sigmoid函数(常见的方法)
2. 逐层训练法(已经很少使用了)

随机梯度下降法/Stochastic Gradient Descent

问题

- 基于梯度下降的误差反向传播反复更新参数
- 但每次更新都用所有训练样本来计算误差$\mathcal{L}$会很慢!

方法：

- 每次更新不用所有训练样本，而是从中随机选取一批(batch)数据来计算误差
- 这批数据称为“mini-batch”，样本的数量称为批大小 (batch size)，如每次选取32个数据样本，64个数据样本
- 通过多次更新参数，“mini-batch”可以覆盖整个训练 数据集，一个epoch称为覆盖一次整个训练数据集

自适应学习率/Adaptive Learning Rate

问题：

- 学习率太大，可能会导致参数更新过大，无法收敛
- 学习率太小，可能会导致参数更新过小，收敛速度慢

方法：

- 自适应学习率算法，如Adam、RMSprop、Adagrad等

```
# Optimizer 优化器
# Stochastic Gradient Descent 随机梯度下降
optimizer = torch.optim.SGD(mlp.parameters(), lr=0.01)
# Adam
optimizer_adam = torch.optim.Adam(mlp.parameters(), lr=0.01)
# RMSprop
optimizer_rmsprop = torch.optim.RMSprop(mlp.parameters(), lr=0.01)
# Adagrad
optimizer_adagrad = torch.optim.Adagrad(mlp.parameters(), lr=0.01)
```

超参数选择/Hyperparameter Selection

首先，我们要分清两个概念：

- 训练集/Training set：用来训练模型的数据集
  - 只用于训练
- 测试集/Testing set：用来评估模型性能的数据集
  - 只用于测试

我们希望训练出一个“好的”模型，即在训练集上表现好，同时在测试集上也表现好。但是，我们不能直接使用测试集来选择超参数，这样是“作弊”行为，并不能增强泛化能力；同时，我们也不能只用训练集来选择超参数，因为这样会导致模型在训练集上过拟合，无法泛化到新的数据。

因此，我们需要将数据集增加一部分：

- 验证集/Validation set：用来选择超参数的数据集

Tip

超参数包括模型和训练的各种设置，例如:

- 神经网络层数
- 每层的神经元数
- 激活函数
- 损失函数
- 批大小
- 训练epoch数
- ...等等

从而，我们可以按照以下步骤选择超参数:

1. 训练集训练模型
2. 在验证集上验证超参数的性能，选择超参数
3. 最终在测试集上测试最终的性能

但是，对于一些稀有数据，拿出一些数据作为验证集会比较浪费

问:如何不需要验证集来选择超参数?

K-Fold Cross Validation

K-Fold Cross Validation是一种常用的方法，它将训练集分成K份，每次取其中一份作为验证集，剩下的K-1份作为训练集，然后重复K次，最后取K次的平均值作为最终的性能。

正则化/Regularization

过拟合原因:

1. 模型过于复杂：如果模型的复杂度（例如神经网络的层数或每层的神经元数）超过了数据的复杂度，那么模型可能会学习到数据中的噪声，而不是真正的模式。这会导致模型在训练集上表现良好，但在新的数据上表现差。
2. 数据量不足：如果可用的数据量太少，模型可能无法从中学习到足够的模式，而是只学习到了特定的样本特征。这也会导致模型在训练集上表现良好，但在新的数据上表现差。
3. 训练时间过长：如果模型训练的时间过长，也可能导致过拟合。因为模型可能会在训练过程中逐渐适应训练数据的特定特征，而忽视了更一般的模式。

解决方法:

1. 增加数据量：增加数据量可以减少过拟合，因为模型可以从更多的数据中学习到更多的模式。
2. 正则化/Regularization

- 正则化是一种用来防止过拟合的方法，它通过在损失函数中加入正则项来限制模型的复杂度，从而提高模型的泛化能力。

数据增强法/Data Augmentation

数据增强（Data Augmentation）是一种在机器学习和深度学习中常用的技术，主要用于扩大训练数据集，以提高模型的性能和泛化能力。数据增强的主要方法包括：

1. 图像数据增强：包括旋转、翻转、缩放、剪切、色彩变换等操作，用于增加图像数据的多样性。
2. 文本数据增强：包括同义词替换、随机插入、随机交换、随机删除等操作，用于增加文本数据的多样性。
3. 音频数据增强：包括改变音调、音量、速度等操作，用于增加音频数据的多样性。

通过数据增强，我们可以在不收集新数据的情况下，有效地增加训练数据的数量，从而提高模型的性能和泛化能力。

提前停止法/Early Stopping

提前停止法（Early Stopping）是一种防止过拟合的策略，主要用于深度学习模型的训练过程中。其基本思想是在训练过程中监控模型在验证集上的性能，当模型的验证性能不再提高时，即停止训练，以防止模型在训练集上过度拟合。

具体操作如下：

1. 在每个训练周期（epoch）结束时，评估模型在验证集上的性能。
2. 如果模型的验证性能在一定数量的训练周期内没有提高，那么停止训练。
3. 使用验证性能最好的模型作为最终模型。

提前停止法的优点是实现简单，且能有效防止过拟合。但需要注意的是，提前停止法需要一个单独的验证集来监控模型的性能，不能使用测试集，以防止信息泄露。

权重衰减/Weight Decay

权重衰减（Weight Decay）是一种常用的正则化技术，主要用于防止神经网络模型过拟合。其基本思想是在模型的损失函数中添加一个与权重大小相关的项，从而限制模型权重的大小。

具体来说，权重衰减的损失函数可以表示为：

$$\mathcal{L}_{\text{total}} = \mathcal{L} + \lambda \|\mathbf{W}\|$$

其中，

- $\mathcal{L}$是原本的损失值
- λ是一个超参数，用来控制正则化的强度
- $\|\mathbf{W}\|$是权重矩阵$\mathbf{W}$的范数，是正则化项

通过权重衰减，我们可以防止模型权重过大，从而防止模型过拟合。同时，权重衰减也可以使模型更加稳定，因为小的权重值意味着模型对输入数据的小变化不会产生大的影响。

Dropout

Dropout 是一种常用的正则化技术，主要用于防止深度学习模型过拟合。其基本思想是在训练过程中随机丢弃（即设置为0）一部分神经元的输出，以增加模型的鲁棒性并防止过拟合。

具体来说，Dropout 的操作如下：

1. 在每个训练周期中，对于每个神经元，以一定的概率（例如0.5）将其输出设置为0。
2. 在测试或验证过程中，不使用 Dropout，但将每个神经元的输出乘以 Dropout 概率，以保持网络的总体输出不变。

Dropout 的优点是实现简单，且能有效防止过拟合。同时，Dropout 也可以被视为一种集成学习方法，因为它相当于在训练过程中对大量不同的网络结构进行了平均。

1. 根据误差反向传播法，当某些隐藏层输出为0，则其对应的梯度为0。也就是说，这会使得只有部分权重会被更新
2. Dropout法可以认为在训练时把一个大网络“拆分”为很多子网络(sub-networks)， 在测试时全部子网络一起“投票”，甚至可以获得更好的结果，这种方法叫做集成学 习(ensemble learning)

卷积神经网络

目的/Motivation

如果我们将图片作为输入，那么每个像素都是一个特征，这样的输入数据维度会非常高，而且每个像素之间的关系也很复杂。传统的全连接神经网络在处理这样的高维数据时，参数数量会非常庞大，计算复杂度高，容易过拟合。

而且，对于图像处理任务，我们通常希望模型能够具有一些特定的性质，比如参数共享、局部感知、平移不变性等。传统的全连接神经网络很难满足这些要求。

同时，一般情况下，识别一张图片并不需要考虑每个像素的绝对位置，而是更关注于像素之间的相对位置。这就意味着我们可以通过一些特殊的操作来减少参数数量，提高模型的效率。

卷积神经网络（Convolutional Neural Networks，CNN）的发展主要是为了解决图像处理中的一些挑战。其主要动机包括：

1. **参数共享**：在传统的全连接神经网络中，每个输入和输出之间都有一个独立的权重，这在处理大型图像时会导致参数数量过多，计算复杂度高，容易过拟合。卷积神经网络通过参数共享，大大减少了模型的参数数量。
2. **局部感知**：在图像处理中，一个像素的信息通常与其邻近的像素密切相关，而与远离的像素关系较小。卷积神经网络通过局部感知机制，使得模型可以专注于局部的图像特征。

3. **平移不变性**：在图像识别任务中，无论目标对象在图像中的位置如何变化，我们都希望模型能够准确地识别出来。卷积神经网络通过卷积操作，实现了对输入的平移不变性。
4. **层次化特征学习**：卷积神经网络通常包含多个卷积层，每个层次可以学习到不同的特征，从低级的边缘和纹理，到高级的部分和整体，这使得模型能够处理复杂的视觉任务。

因此，卷积神经网络的发展是为了更好地处理图像和其他类似的高维数据，提高模型的性能和效率。

卷积算法/Convolutional Algorithm

卷积神经网络的核心是卷积操作，它是一种特殊的线性操作，用于提取输入数据的局部特征。卷积操作的基本思想是通过滑动一个卷积核（也称为滤波器）在输入数据上，计算卷积核与输入数据的内积，从而得到输出数据。

1-D向量的卷积

$$\mathbf{s}_t = (\mathbf{x} * \mathbf{w})_t = \sum_{k=-\infty}^{+\infty} \mathbf{x}_k \mathbf{w}_{t-k}$$

2-D灰度图片的边缘检测

卷积核：（检测水平边缘）

$$\mathbf{w} = \begin{pmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{pmatrix}$$

$$\mathbf{s}_{r,c} = ((\mathbf{x} * \mathbf{W}))_{r,c} = \sum_{i=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} \mathbf{x}_{r,c} \mathbf{w}_{r-i,c-j}$$

2-D彩色图片的卷积

卷积核：（检测红色通道的水平边缘）

$$\mathbf{W} = \begin{pmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{pmatrix}$$

$$\mathbf{s}_{r,c} = ((\mathbf{x} * \mathbf{W}))_{r,c} = \sum_{i=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} \mathbf{x}_{r,c} \mathbf{w}_{r-i,c-j}$$

- 卷积核形状Kernel/Filtershape
  - 对RGB图像进行卷积需要3个channel
  - 形状 (filter_height × filter_width × input_channels × n_filters)
- 卷积核数量
  - 每个滤波器有一个对应的输出特征图。

    - 输出的channels == 卷积核数量
    - 输出形状(feature_height × feature_width × n_filters)
        - 根据input size, padding, strides, filter size决定

感受野/Receptive Field

感受野（Receptive Field）指的是在输入数据中，能影响到当前神经元输出的所有区域。换句话说，就是当前神经元“能看到”的输入数据的范围。

在卷积神经网络的第一层，一个神经元的感受野大小通常就是卷积核的大小。但在更深的层次，由于每一层的输出都是上一层的卷积结果，所以一个神经元的感受野会包括上一层多个神经元的感受野，因此感受野的大小会随着网络深度的增加而增大。

理解感受野的概念对于理解卷积神经网络的工作原理和设计网络结构非常重要。例如，如果我们希望网络能够捕捉到更大范围的上下文信息，我们可能需要增加网络的深度或者使用更大的卷积核，以增大神经元的感受野。

空洞卷积

空洞卷积，把卷积核中间加入空洞，使得卷积核的感受野变大，从而提高网络的感知能力。

原始卷积核：

$$\mathbf{W} = \begin{pmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{pmatrix}$$

空洞化：

$$\mathbf{W} = \begin{pmatrix} -1 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 2 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & -1 \end{pmatrix}$$

3-D卷积

在3D体素上进行卷积

- 卷积核(滤波器)的形状
    - 对MRI 图像进行卷积, 每个滤波器有1channel
    - 滤波器形状 (filter_depth × filter_height × filter_width × input_channels × n_filters)
- 滤波器数量(channel数)
    - 每个滤波器对于一个对应的输出的特征体素(feature volume)
    - 输出channel数 == 滤波器数量
    - 输出的形状(feature_depth × feature_height × feature_width × n_filters)

池化算法/Pooling Algorithm

目的：增强图像的平移不变性

- 小的平移变化不应该对网络的输出结果产生显著影响

1-D向量的池化

- 最大池化/Max Pooling

$$\begin{pmatrix}1 & 3 & 3 & 0 & 1 & 2 & 0\end{pmatrix} \mapsto \begin{pmatrix}3 & 3 & 2\end{pmatrix}$$

- 平均池化/Average Pooling

$$\begin{pmatrix}1 & 3 & 3 & 0 & 1 & 2 & 0\end{pmatrix} \mapsto \begin{pmatrix}0.77 & 1.33 & 1\end{pmatrix}$$

2-D矩阵的池化

- 最大池化/Max Pooling
- 平均池化/Average Pooling

空间金字塔池化

空间金字塔池化（Spatial Pyramid Pooling，简称SPP）是一种特殊的池化方法，它在卷积神经网络（CNN）中被用来处理任意大小或者比例的输入图像。

在传统的CNN中，输入图像必须被调整到固定的大小才能被处理。然而，这种方法可能会导致一些空间信息的丢失。为了解决这个问题，SPP在网络的卷积部分后面引入了一个池化层，这个池化层可以生成固定长度的特征向量，无论输入图像的大小或者比例如何。

SPP的工作原理是将输入图像划分为多个不同大小的区域，然后在每个区域内进行池化操作（例如最大池化或者平均池化）。这些区域的大小是按照金字塔的形式来确定的，也就是说，越靠近金字塔顶部的区域越小，越靠近金字塔底部的区域越大。最后，所有区域的池化结果被连接起来，形成一个固定长度的特征向量。

SPP的主要优点是它可以处理任意大小或者比例的输入图像，而不需要进行图像的缩放或者裁剪。此外，它还可以保留图像的空间信息，从而提高模型的性能。

分层表示学习/Hierarchical Representation Learning

分层表示学习（Hierarchical Representation Learning）是一种机器学习方法，它的目标是学习数据的多层次结构或者表示。这种方法通常在深度学习模型中使用，例如卷积神经网络（CNN）和递归神经网络（RNN）。

在分层表示学习中，模型的每一层都会学习数据的一种表示。在底层，模型可能会学习一些简单的特征，例如在图像处理中，底层可能会学习边缘、颜色和纹理等特征。在中间层，模型可

能会学习一些更复杂的特征，例如形状和部分物体。在顶层，模型可能会学习一些高级的特征，例如整个物体。

分层表示学习的主要优点是它可以自动地从数据中学习有用的特征，而不需要人工设计。此外，由于模型的每一层都会学习数据的一种表示，这使得模型可以处理复杂的数据结构，从而提高模型的性能。

补充：图像的基本知识

图像的基本单位是像素（Pixel），每个像素代表图像中的一个点，它包含了图像的颜色和亮度信息。图像的大小通常用宽度（Width）和高度（Height）来表示，例如一个 640x480 的图像表示宽度为 640 像素，高度为 480 像素。

一般地，我们可以使用矩阵来表示图像，其中矩阵的每个元素代表一个像素的颜色值。在灰度图像中，每个像素的颜色值通常是一个介于 0 到 255 之间的整数，表示像素的亮度。在彩色图像中，每个像素的颜色值通常是一个包含红色、绿色和蓝色三个通道的三元组，分别表示像素的红色、绿色和蓝色分量。

每个像素的值域可以用bit表示，比如8bit的灰度图像，每个像素的值域是0-255，也就是2^8=256种颜色。而24bit的彩色图像，每个像素的值域是0-255，也就是2^24=16777216种颜色。

卷积神经网络结构/Convolutional Architectures

卷积神经网络（Convolutional Neural Networks，CNN）是一种专门用于处理图像和其他高维数据的神经网络模型。它的核心是卷积层（Convolutional Layer）和池化层（Pooling Layer），通过这些层的堆叠，CNN可以学习到数据的多层次特征表示。

转置卷积(反卷积) /Transposed Convolutional Algorithm

目的：卷积和池化只能保持或缩小特征图的尺寸

卷积神经网络的应用

目标检测

目标检测（Object Detection）是计算机视觉中的一个重要任务，它的目标是在图像中找到特定目标的位置，并对这些目标进行分类。

具体来说，目标检测算法通常会输出一个边界框（Bounding Box），这个边界框标记了目标在图像中的位置。同时，算法还会输出一个类别标签，表示这个目标属于哪个类别。

目标检测与图像分类的主要区别在于，图像分类只需要确定图像的整体类别，而目标检测需要找到图像中每个目标的位置和类别。因此，目标检测的任务更为复杂。

目标检测在许多领域都有广泛的应用，包括自动驾驶、视频监控、医疗图像分析等。

Question

如何评估效果?

我们引入交并比：Intersection over Union (IoU)

设ground truth为B_{gt}，预测框为B_{pred}，则交并比定义为：

$$IoU = \frac{Area(B_{gt} \cap B_{pred})}{Area(B_{gt} \cup B_{pred})}$$

如何通过IoU评价目标检测的效果?

- 当IoU等于1时，意味着预测边界框与真实边界框完全重合， 预测结果是完美的。
- 当IoU等于0时，表示预测边界框与真实边界框完全没有重叠， 预测结果完全不匹配。
- 当IoU大于某个阈值(如0.5)时，我们通常认为物体被成功 检测到。在实际应用中，可以根据任务需求调整IoU阈值， 以权衡检测的精确度和召回率。

同时，我们也能定义平均精确度：Average Precision (AP)

首先我们定义下面四个概念：

- 真阳性(True Positive，TP):预测为正类且实际上是正类的样本数。
- 假阳性(False Positive，FP):预测为正类但实际上是负类的样本数。
- 真阴性(True Negative，TN):预测为负类且实际上是负类的样本数。
- 假阴性(False Negative，FN):预测为负类但实际上是正类的样本数。

精度率：

$$Precision = \frac{TP}{TP + FP}$$

召回率：

$$Recall = \frac{TP}{TP + FN}$$

从而我们可以画出Precision-Recall曲线，计算AP，公式为

$$AP = \int Precision(Recall)\mathrm{d}Recall$$

非极大值抑制/Non-Maximum Suppression

非极大值抑制（Non-Maximum Suppression，NMS）是目标检测中常用的一种技术，用于去除重叠的边界框，保留最可能的目标边界框。

具体来说，NMS的工作原理是：

1. 对所有边界框按照置信度进行排序，从高到低。
2. 选择置信度最高的边界框，将其添加到最终的结果中。
3. 计算其他边界框与当前边界框的交并比（IoU），如果大于一定阈值（例如0.5），则将其删除。
4. 重复步骤2和步骤3，直到所有边界框都被处理。

例子：R-CNN

- Step1:使用选择性搜索算法 (Selective Search)从图像中提取约 2,000个候选区域。
- Step2:将所有候选区域调整为给定的固定大小。
- Step3:将每个候选区域输入VGG网络以提取图像特征，并通过特征对图像进行分类。
- Step4:通过回归获得边界框的位置。

例子：SPP Net

R-CNN的局限:

1. 选择性搜索速度慢。
2. 调整候选区域的尺寸会导致宽度和高度的比例变化，从而影响分类的准确性。
3. 将每个区域单独输入到VGG中，这个过程非常慢。
4. 非端到端训练。

SPP Net 的改进:

1. 解决了R-CNN的第二和第三个局限性。
2. 提出了空间金字塔池化(Spatial Pyramid Pooling，SPP)，将特征调整为相同的尺寸。
3. 将整个图像输入到CNN中以获得全局特征，并在全局特征上获得每个区域的特征。
4. 使用全局特征，而非逐个将图像输入到VGG中。
5. 不改变图像尺度。

例子：Fast R-CNN

SPP Net局限性:

1. 仍然使用选择性搜索，速度非常慢。
2. 仍然不是端到端训练。

Fast R-CNN的贡献:

1. 提出了感兴趣区域(Region of Interest，ROI)池化层，这是一种简化的空间金字塔池化方法，只使用一个池化尺寸。
2. 端到端训练(近似)，分类器和边界框回归器与CNN特征提取器一起进行训练。

例子：Faster R-CNN

Fast R-CNN的局限性:

1. 仍然使用选择性搜索，速度非常慢。

Faster R-CNN的贡献:

1. 使用区域提议网络(Region Proposal Network, RPN)替换选择性搜索，使神经网络能够搜索候选区域，速度更快。
2. 实现真·端到端训练，准确性提高。

目标检测:重新思考

上述算法都需要进行区域提议，但这一步骤是否必要呢?

为了知道图像中不同物体的位置，人类不需要手动选择许多区域并逐个进行分类，相反，人类可以通过一次看图就发现图像中的所有物体。因此，重新考虑目标检测算法中是否真的需要进行区域提议的步骤。

我们把目标检测问题抽象成一个回归问题。

- 不需要区域提议，完全卷积网络(Fully Convolutional Neural Network，FCN)可以直接输出类别标签和位置信息。
- 速度非常快

例子：YOLO

- 一张图像被划分成一个7x7的网格，每个网格单元都会预测对应的图像位置的类别标签、物体宽度和高度，以及物体置信度。
- 每个网格单元都有30个输出，其中10个用于存储2个边界框(x，y，w，h，置信度)的信息，另外20个则用于存储VOC数据集中20个类别的概率。因此，每 个网格单元可以有2个可能的边界框，但最终只能匹配到一种物体。
- 去除置信度低的边界框，并执行非极大值抑制(NMS)以获取最终结果。

例子：YOLOv2

YOLO算法的局限性:

- 难以检测小物体。
  YOLO v2算法:

1. 为每个网格单元预定义多个提议区域，即预定义的先验框。
2. 将网格分辨率从7x7提高到 13x13。
3. 为每个网格单元预定义6个先验框，YOLOv2能够预测13x13x6=1014个潜在边界框，而YOLOv1只能预测7x7x2=98个边界框。
4. 每个网格单元有多个类别概率向量，即每个网格单元可以检测不同类型的物体(而YOLOv1中每个网格单元只能是一个类别)

图像分割

语义分割（Semantic Segmentation）：它的目标是理解图像在像素级别上的含义。具体来说，语义分割不仅要划分出图像中的各个物体，还要识别出这些物体的类别。例如，在一张包含猫和狗的图像中，语义分割不仅要将猫和狗分割出来，还要标记出哪些像素属于猫，哪些像素属于狗。

实例分割（Instance Segmentation）：这是一种更高级的分割任务，它不仅要像语义分割那样识别出每个像素的类别，还要区分出属于同一类别的不同实例。例如，在一张包含多只猫的图像中，实例分割不仅要标记出哪些像素属于猫，还要区分出每一只猫。

算法：全卷积神经网络/FCN

全卷积神经网络（Fully Convolutional Network，简称FCN）是一种用于语义分割的深度学习模型。它的主要特点是，网络中所有的层都是卷积层，没有全连接层。

全卷积神经网络的主要优点是可以接受任意大小的输入图像，并且在整个图像上共享参数，这使得它在处理大型图像时具有很高的效率。

FCN的基本结构包括两部分：卷积层和上采样层。卷积层用于提取图像的特征，上采样层用于将卷积层输出的低分辨率特征图恢复到输入图像的原始分辨率，以便进行像素级别的预测。

FCN的一个重要特性是它可以进行端到端的训练和预测，这意味着它可以直接从输入图像预测出每个像素的类别，无需其他后处理步骤。

全卷积神经网络（FCN）的优点包括：

1. **端到端训练和预测**：FCN可以直接从输入图像预测出每个像素的类别，无需其他后处理步骤，这使得训练和预测过程更加简单和直接。

然而，FCN也有一些缺点：

1. **上采样过程可能导致细节丢失**：在FCN中，卷积层输出的低分辨率特征图需要通过上采样层恢复到输入图像的原始分辨率。这个过程可能会导致一些细节信息的丢失。
2. **无法处理实例分割任务**：FCN只能进行语义分割，无法区分出属于同一类别的不同实例。

所以我们发展出了跳跃连接的算法，如U-Net，用于解决FCN的缺点。

在跳跃连接中，前面层的输出会直接连接到后面层的输入，这样在反向传播过程中，梯度可以直接通过这个连接传播，避免了梯度消失或爆炸的问题。

此外，跳跃连接还有助于保留低层特征信息。在一些任务中，如图像分割和物体检测等，低层的特征信息（如边缘、纹理等）是非常重要的。通过跳跃连接，这些低层的特征信息可以直接传递到网络的高层，从而提高模型的性能。

图像分割中的目标函数

像素级的交叉熵损失函数（Pixel-wise Cross-Entropy Loss）是图像分割任务中常用的目标函数。

- 像素级的交叉熵损失将每个像素看作一个单独的标签进行分类
- 缺点:面积较大的物体对损失的权重较大，导致对于分割小物体的性能较差

Dice损失函数是一种常用的用于图像分割任务的损失函数，它可以解决像素级的交叉熵损失函数的缺点。

- 解决了像素交叉熵中不平衡的问题，可以看着是可求导的IoU
- $Dice = \frac{2|A \cap B|}{|A|+|B|}$, 其中A和B是值为0或1的两个向量
  - $A \cap B$是交集面积
  - 如果A和B完全重叠, Dice=1
  - 如果A和B完全分开, Dice=0

人脸识别/Face Recognition

首先，我们可以将人脸识别分成三个步骤：

1. 检测图像中的人脸位置/Face Detection
2. 将人脸对齐到图像中心/Face Alignment
3. 对图像进行人脸身份识别/Face Identification

Face Recognition通常分为两类

- 人脸验证(Face Verification)：给定两张图像，判断它们是否属于同一人。该任务旨在验证两个不同图像中的人是否为同一人，而不需要知道其身份标识。
- 人脸识别(Face Identification)：给定一张图像，将其与已知的人脸库进行比较，从而确定图像中人脸对应的身份标识(ID)。该任务旨在从多个身份中识别出单个人的身份。

同时，问题还能从另一个方面分为两类

- Close-set Face ID：输入的人脸在训练集中，即一个简单的分类问题。
- Open-set Face ID：输入的人脸不在训练集中。

我们一般都是使用Open-set Face ID开放集人脸识别，其原理是:

- 预先训练一个图像编码器，从图像中提取区分性特征向量(Discriminative Features Vector)。
- 不同人的脸有非常不同的特征向量，而相同人的Feature Vector即使图像有不同的光照条件也很相似。
- 通过比较输入的待查询特征向量与数据库中的已知特征向量的距离来找到人脸ID。
- Close-set Face Identification: 是一个简单的分类问题，与MNIST类似。给定一张图像，输出其对应的类别标签。

- Close-set Face Verification: 也是一个简单的分类问题。给定两个图像，输出两个类别标签并比较这两个标签是否相等。
- Open-set Face Identification: 是一种特征提取问题，它首先提取数据集中所有图像的特征向量，然后在输入一个图像时，查找具有最佳匹配特征向量的ID。
- Open-set Face Verification: 也是一个特征提取问题，它提取两个图像的特征向量并计算相似度分数。如果分数高于阈值，则这两个图像是同一个人。

对抗神经网络/Generative Adversarial Networks

生成式模型

- 计算机图形学
  - 完全基于先验知识
  - 难以扩展和泛化
  - 开发耗时
- 机器学习/深度学习
  - 减少先验知识的需求
  - 从数据中学习
- 统计/深度生成模型
  - 仍需要一些先验知识...
  - 损失函数、学习方法、架构、先验分布(例如，高斯)

Notation.

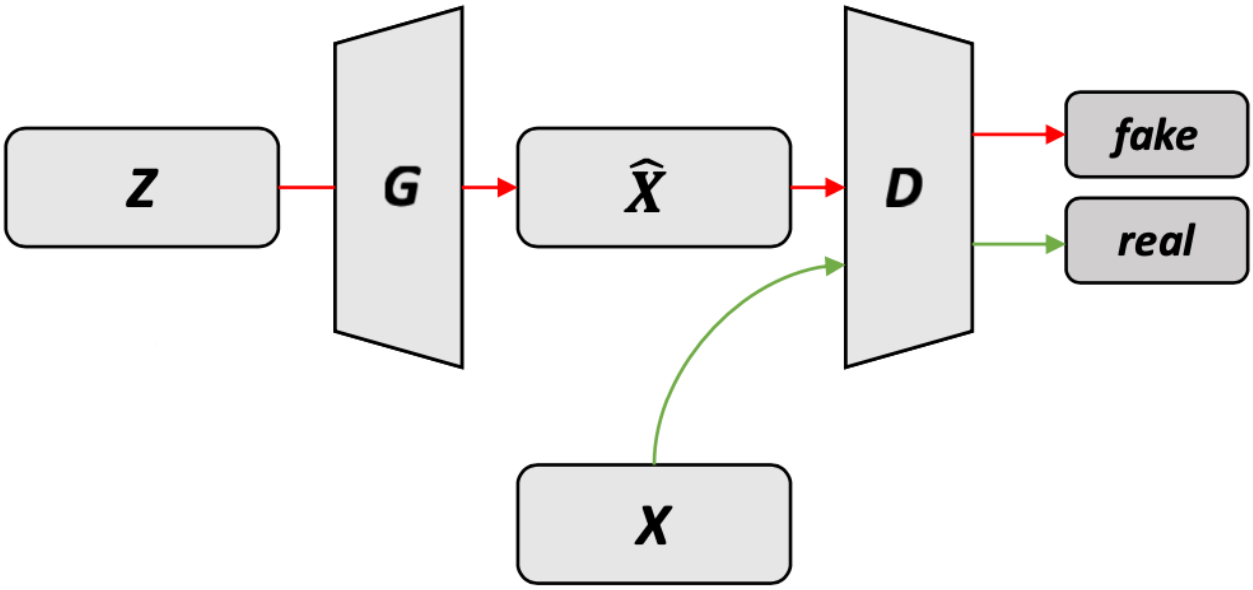
- $x^j \sim p_{\text{data}}(j = 1, \cdots, |\mathcal{D}|)$: 真实数据
- $\mathcal{D}$: 真实数据集
- p_{data}: 真实数据分布
- $\theta \in \mathcal{M}$: 参数空间

判别式 v.s. 生成式

- 判别式模型: 分类数据，寻找决策边界$P(Y|X)$
- 生成式模型: 生成数据，寻找联合分布$P(Y, X)$

朴素GAN/Vanilla GAN

结构:



目标函数:

$$
\begin{aligned}
\min_G \max_D V(D,G) &= \min_G \max_D \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}}[\log(1-D(G(\mathbf{z})))] \\
\mathcal{L}_D &= -\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] - \mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}}[\log(1-D(G(\mathbf{z})))] \\
\mathcal{L}_G &= -\mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}}[\log(1-D(G(\mathbf{z})))]
\end{aligned}
$$

循环神经网络

词的表示/Word Representation

One-hot Vector

- 循环神经网络逐个接收单词，但让我们看看计算机中如何表示一个单词。
- 简单方法1:One-hot Vector

"Deep Learning is very useful"

- "Deep": [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- "Learning": [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
- "is": [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
- "very": [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
- "useful": [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]

缺点:

- 大词汇量将导致“维数灾难”
- 所有单词表示都是独立的! 太稀疏了!

词袋模型/Bag of Words

- 简单方法2:词袋模型

- 使用单词频率来表示句子

"we like PKU, don't we?"

- "we": 2
- "like": 1
- "PKU": 1
- "don't": 1

缺点:

- 大词汇量将导致“维数灾难”。
- 丢失了单词位置的信息。

词嵌入/Word Embedding

- 用一组浮点数向量来表示一个单词。

理想的词嵌入:

- 低维度 <=> 用高级特征表示单词
- 能够包含单词的语义信息

我们可以通过**习得词嵌入**来获得这些特征。

- 现有的算法通过阅读大量文本文档来学习词嵌入表，以发现其中的模式，这是一种自监督(self-supervised)学习方法

Word2Vec

- 连续词袋模型(Continuous Bag-of-Words，CBOW): 根据上下文预测中间的词。
- 在“a blue bird on the tree”这句话里, “bird” 的上下文是 [“a”, “blue”, “on”, “the”, “tree”]
- Skip-Gram(SG)与CBOW相反，但目的相同
- CBOW通过上下文预测中间词，而SG则通过中间词预测上下文。
- 在句子“a blue bird on the tree”中，SG 的输入是 “bird”, 输出是are [“a”, “blue”, “on”, “the”, “tree”]
- 噪声对比估计(NCE)
- Skip-Gram有多个目标输出，因此我们使用Sigmoid而不是Softmax。
    - 词汇表中的每个单词都被分为正样本和负样本，并且我们独立地对每个单词进行分类。
- 大词汇表将导致巨大的计算成本
    - 我们使用负采样来加速损失函数的计算，从词汇表中随机采样N个负样本。
- 这种方法被称为噪声对比估计(Noise-Contrastive Estimation):

$$\mathcal{L}_{\mathrm{SG}} = -\left(\sum_{i \in \text{Positive samples}} \log(y_i) + \sum_{j \in \text{Randomly select N negative samples}} \log(-y_j) \right)$$

序列数据/Sequential Data

朴素循环神经网络/Vanilla Recurrent Neural Network

长短期记忆网络/LSTM Long Short-Term Memory

时间序列应用/Time-series Applications

Transformer

动机

早期序列建模方法具有局限性:

- RNN(循环神经网络): RNN可以处理序列数据，但在处理长序列时存在梯度消失与梯度爆炸问题。
- LSTM(长短时记忆)与GRU(门控循环单元): 为解决RNN的梯度问题，引入了更复杂的结构，如LSTM和GRU，但计算效率相对较低。
- CNN(卷积神经网络): 虽然主要用于图像处理，但也可以应用于序列数据，如文本分类。然而，CNN的感受野有限，不能直接捕捉全局依赖关系。

长度依赖问题!

人类视觉注意力的特点:

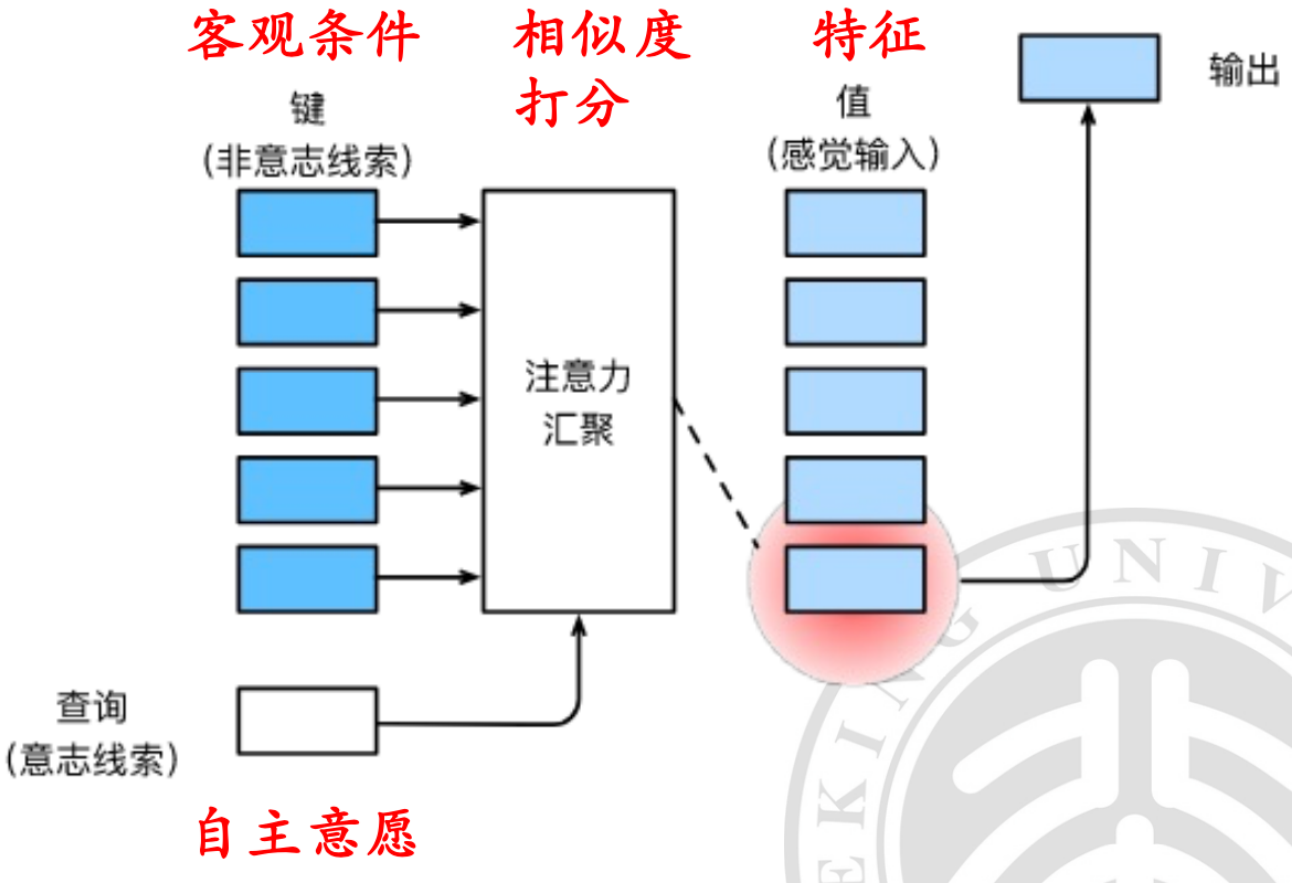
- 有限的处理能力，而忽略无关的信息。
  - 稀有资源
- 空间选择性，人类视觉注意力可以在 空间中选择性地关注特定位置的信息。
- 特征选择性，人类视觉注意力还可以 根据特征(如颜色、形状、运动等) 选择性地关注信息。

自注意力机制

注意力提示: 查询、键和值

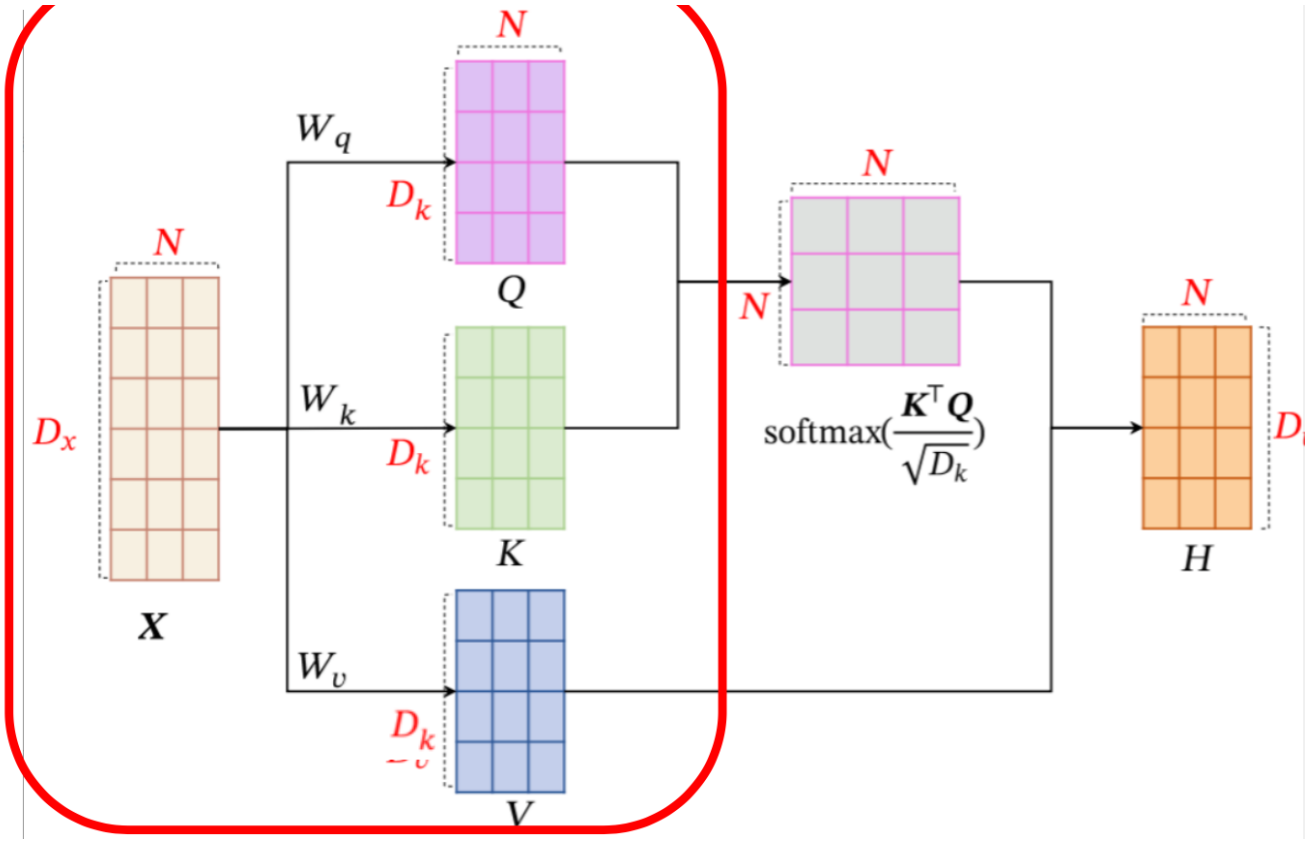
- 非自主性提示: 键(Key)、值(Value)
- 自主性提示: 查询(Query)
- 注意力汇聚: 打分函数，注意力权重矩阵，加权平均

$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$$



词嵌入到Q、K、V

- Q、K、V是三个不同的空间
- 我们首先将输入序列转换为嵌入表示，然后，通过三个不同的线性层投影W_q、W_k和W_v，分别生成查询矩阵(Q)、键矩阵(K)和值矩阵(V)。这三个矩阵将用于计算注意力输出。
  - N是序列长度
  - D_x是嵌入的特征维度
  - D_k是QKV矩阵的维度，也叫head dim



最终，这个网络的输出是：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{K^\top Q}{\sqrt{D_k}}\right)V$$

打分函数:缩放点积(Scaled Dot-Product)

```
Attention(Q, K, V):
        return softmax(K^T * Q / sqrt(d_k)) * V
```

这种注意力汇聚方法采用点积计算Q和K之间的相似度，然后将结果除以缩放因子以确保梯度稳定。之后，对相似度矩阵应用softmax函数以归一化，得到注意力权重矩阵，最后将权重与V相乘并求和。

- `Attention(Q, K, V)`：表示注意力汇聚的输出结果。
- `Q`：表示查询(Query)矩阵
- `K`：表示键(Key)矩阵
- `V`：表示值(Value)矩阵
- `d_k`：表示键向量的维度
- `*`：表示矩阵乘法
- `^T`：表示矩阵转置
- `softmax`：表示应用 `softmax` 函数，对结果进行归一化
- `sqrt`：表示平方根

其中，为了量查询向量与键向量之间的相似度(以确定输入序列中哪些部分与给定的查询相关)

- 我们首先计算查询向量和键向量之间的点积$K^\top Q$，然后将结果除以缩放因子$\sqrt{D_k}$
- 缩放与归一: 得到注意力权重矩阵
  - 缩放: 将每个点积结果除以一个缩放因子，通常是QKV维度的平方根。这样做的目的是防止点积结果过大，导致接下来的`softmax`操作中梯度过小。
  - `softmax`归一化: 对计算得到的缩放点 积矩阵应用softmax函数。这一步将点积值转换为概率分布，使得所有注意力权重的和为1。
- 应用注意力汇聚: 加权求和，将注意力权重矩阵与值矩阵(`V`)相乘。

打分函数:加性注意力(Additive Attention)

- 加性注意力是一种用于计算查询和键之间相似度的方法。它首先将查询和键分 别经过线性变换(全连接层)，然后将两者相加，并通过非线性激活函数(如 tanh)计算相似度。接下来，使用可学习的权重向量计算最终的相似度分数。

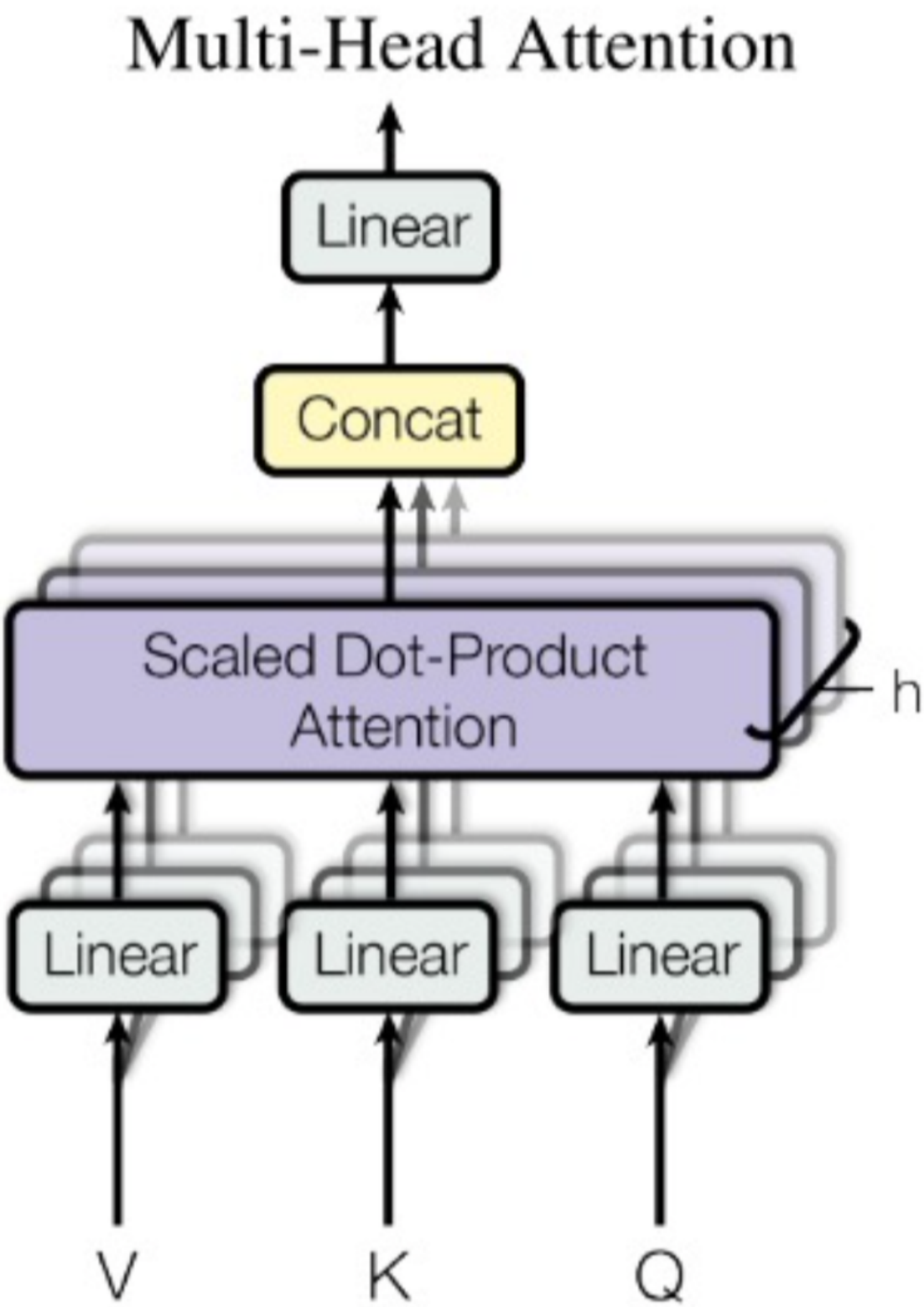
$$Attention(Q, K.V) = \text{softmax}\left(\tanh(Q * W_q + K * W_k) * V_a^\top\right) * V$$

- `Attention(Q, K, V)`: 表示注意力汇聚的输出结果。
- `Q`: 表示查询(Query)矩阵
- `K`: 表示键(Key)矩阵
- `V`: 表示值(Value)矩阵
- `W_q`: 表示查询的线性变换矩阵
- `W_k`: 表示键的线性变换矩阵
- `V_a`: 表示可学习的权重向量
- `+`: 表示矩阵相加
- `tanh`: 表示应用`tanh`非线性激活函数

多头注意力

多头注意力(multihead attention)

- `h`个self-attention组合
- 映射到`h`个空间，关注不同的特征维度
- 将这`h`个注意力汇聚的输出拼接在一起，并且通过另一个可以学习的线性投影进行变换，以产生最终输出



对于一个输入序列，假设其长度为 `N`，嵌入维度(特征维度)为 `D`，头数为 `h`（`num_heads`），则对于一个多头注意力(multi-head attention)计算过程中各项的形状如下:

- 输入矩阵:
  - `Q=K=V`，形状为: `(N, D)`
- 对于一共 `h` 个注意力头，`i` 表示注意力头的索引
- 每个注意力头独立的QKV权重矩阵:
  $W_{Q_i}, W_{K_i}, W_{V_i}$，形状为: `(D, head_dim)`

其中`head_dim = D/h`

- 线性变换后的查询、键和值矩阵:
  - Q_i, K_i, V_i，形状为: `(N, head_dim)`
- 缩放点积注意力计算中的相似度矩阵:
  - $Q_i * K_i^\top$，形状为: `(N, N)`
- 缩放点积注意力的输出:
  - $\text{Attention}_i(Q_i, K_i, V_i)$, 形状为: `(N, head_dim)`
- 多头注意力的拼接输出:
  - `Concat_Attention`, 形状为: `(N,D)`
  - 因为我们拼接了`h`个头, 每个头的维度为`head_dim`, 总维度为`h * head_dim = D`
- 线性变换后的输出矩阵:
  - W_0，形状为: `(D, D)`
- 多头注意力的输出:
  - $\text{Multihead_Attention}(Q, K, V)$, 形状为: `(N, D)`

位置编码与掩码

位置编码(Positional Encoding)

回忆: 词嵌入和RNN

- Embedding序列
- 句子 = 词 + 位置顺序排列
- 词信息 >>> 词嵌入
- 位置信息 >>> 位置嵌入
- 表达句子的向量序列 = 词嵌入向量 + 位置嵌入向量

对于一个序列长度为N，嵌入维度为d_{model} 的输入，位置嵌入矩阵的形状为(N, d_{model})。每个位置嵌入向量的形状为$(d_{\text{model}},)$

$$PE(pos, 2i) = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE(pos, 2i+1) = cos(pos/10000^{2i/d_{\text{model}}})$$

表示位置嵌入矩阵的第pos行、第$2i$列的值
表示位置嵌入矩阵的第pos行、第$2i+1$列的值。

- i:是嵌入向量的索引
- d_{model}:嵌入向量的维度，pos是位置编码的位置。

掩码(Mask)

防止在输入序列中的某些位置上的信息泄露到当前位置上。

- 在语言建模任务中，需要在预测当前词时，只利用前面的词，而不使用后面的词。

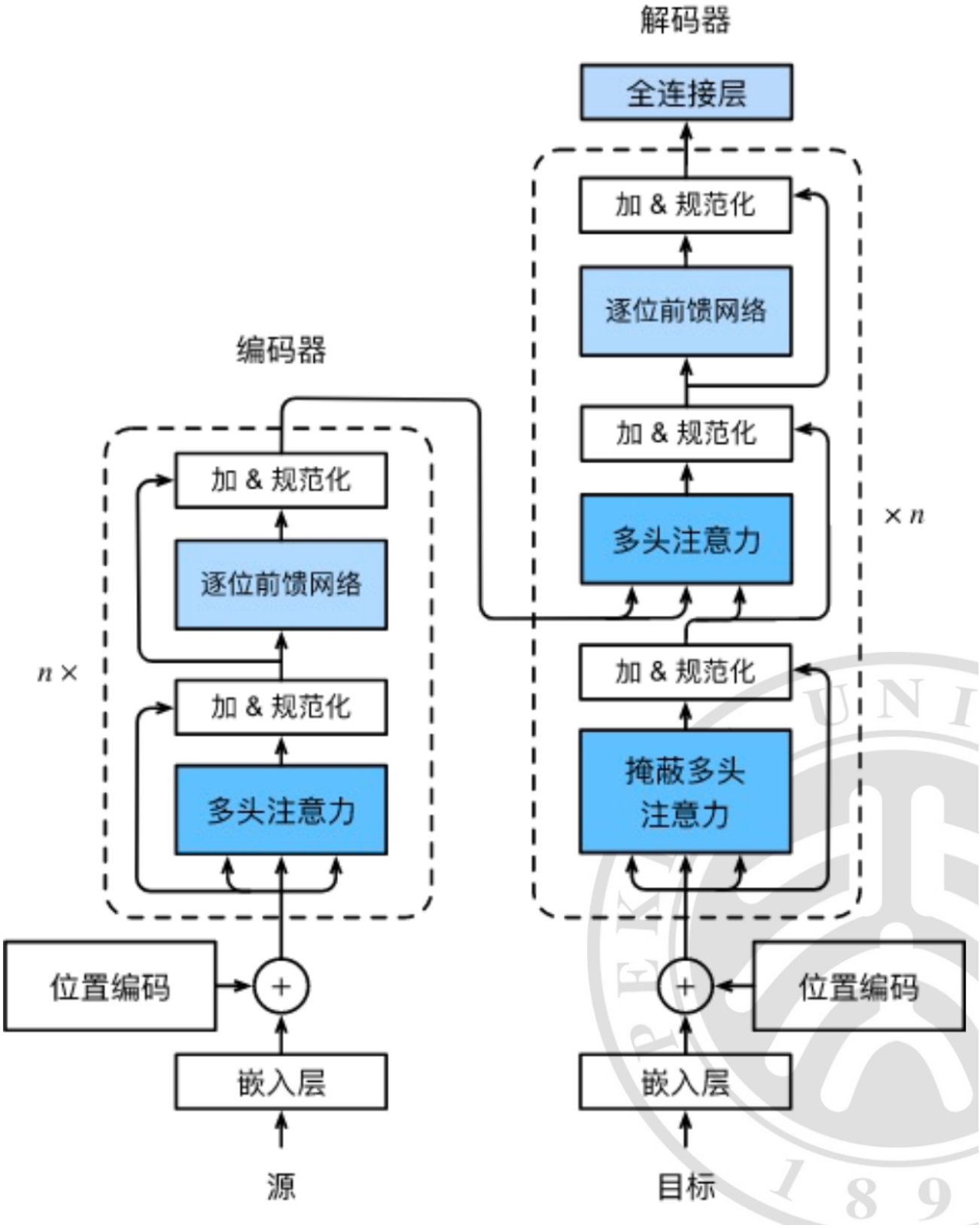
- 创建一个掩码矩阵，形状为`(N,N)`，其中`N`是输入序列的长度。
- 对于掩码矩阵中的每个元素，如果该元素对应的位置是要掩盖的位置，则将其设置为负无穷(`-inf`)，否则将其设置为0。
- 在`softmax`计算之前，将掩码矩阵与注意力分数相加，这样，被掩盖的位置的注意力分数就变成了负无穷，`softmax`之后，它们对应的注意力权重就会变成0，即不会对当前位置产生影响。

$$\text{Attention}(Q, K, V, \text{mask}) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{mask}\right)V$$

Transformer

整体架构

- 整体结构由编码器(Encoder)和解码器(Decoder)两部分组成
- 编码器和解码器都是由多个相同的层组成，每层中包含了一个多头注意力机制和一个全连接前馈网络，这些层之间通过残差连接和层归一化(Layer Normalization)相连。



计算过程: 编码器

1. 源输入序列经过嵌入层的处理得到词嵌入，并加入位置编码，得到最终的输入向量
2. 通过多头注意力的h组线性层对词嵌入表示进行变换，将其映射为三个不同的空间，得到查询矩阵Q、键矩阵K和值矩阵V
3. 计算缩放点积注意力输出。使用Q、K、V三个矩阵，按照缩放点积注意力的计算公式进行计算，得到输出矩阵。这个输出矩阵包含了每个词向量对于其他所有词向量的注意力权重

计算过程: 残差连接和加&层规范化(AddNorm)
4. 将输入向量和注意力输出向量相加，得到残差连接的结果
5. 对残差连接的结果进行层规范化(Layer Normalization)
- 将每层的输出值归一化到均值为0、方差为1的范围内
- 层规范化将每个神经元的输出值x减去它在这一层的均值μ，然后除以标准差σ，最后乘以可学习的缩放因子α
- 层规范化的公式如下:

$$\mathrm{LayerNorm}(x) = \alpha \odot \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta$$

计算过程: 逐位前馈神经网络
6. 逐位前馈网络(Feed-Forward Network, FFN)用 于在自注意力机制之后进行非线性变换。计算过程如下

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

- 其中，
  - x表示输入向量
  - W_1和b_1是第一个全连接层的权重矩阵和偏置向量
  - W_2和b_2是第二个全连接层的权重矩阵和偏置向量

逐位前馈网络由两个全连接层和一个 `ReLU` 激活函数组成

计算过程: 多个编码器堆叠

$$\mathrm{Transformer}(x) = \mathrm{Encoder}_n(\cdots \mathrm{Encoder}_2(\mathrm{Encoder}_1(x)))$$

7. 如果有n个编码器，则可以将它们依次串联起来。其中，第一个编码器的输入是词嵌入向量加上位置编码向量
8. 最后一个编码器的输出就是最终的编码表示，它将用于传递给解码器进行下一步的处理

计算过程:解码器
encoder的输出并没直接作为decoder的直接输入

- 解码器的输入被称为目标序列。初始 `decoder` 的 `time step` 为1时(也就是第一次接收输入)，其输入为一个特殊的 `token`，即目标序列开始的 `token` (如 `<BOS>`)，也可能是其它视任务而定的输入等等，其目标则是预测翻译后的第1个单词(`token`)是什么;
- 然后 `<BOS>` 和预测出来的第1个单词一起，再次作为 `decoder` 的输入，得到第2个预测单词; 后续依此类推。直到遇到特殊的结束 `token` (如 `<EOS>`)或者达到最大输出长度为止。

9. 目标序列嵌入和位置编码: 与编码器类似，目标序列首先经过嵌入层的处理得到词嵌入，并加入位置编码，得到最终的输入向量
10. 通过掩蔽多头注意力层(masked multi-head attention)生成解码器自注意力矩阵，用于捕捉当前解码器状态与之前解码器状态的依赖关系。具体计算过程与编码器的自注意力矩阵计算过程类似
    - Mask机制屏蔽了未来的序列信息

11. 接下来编码器-解码器注意力层(encoder-decoder attention)将编码器输出的信息有效地融入到当前解码器状态中，以帮助解码器更好地进行下一步预测
    - 在编码器-解码器注意力中，Q来自前一个解码器层的输出，而K和V来自编码器的输出。
12. 接下来逐位前馈网络和 `AddNorm` 与解码层一致。
13. 堆叠n个解码器，前一个解码器的输出和对应的编码器的输出是当前解码器的输入
    - 第1个解码器层:

$$\begin{aligned}\mathrm{Out}_0 &= \mathrm{PositionEncoding}(Y)\\ \mathrm{Out}_1 &= \mathrm{decoder}_1(\mathrm{Out}_0, E)\end{aligned}$$

    - 第i个解码器层($1 < i \leq n$):

$$\mathrm{Out}_i = \mathrm{decoder}_i(\mathrm{Out}_{i-1}, E)$$

    最后，我们可以对最后一个解码器层的输出施加 `softmax` 函数来计算词汇表的概率分布:

$$P_{\mathrm{vocab}}(y_i|y_1, \cdots, y_{i-1}, x) = \mathrm{softmax}(W_{\mathrm{out}} * \mathrm{Out}_n + b_{\mathrm{out}})$$

常用预训练模型

预训练模型是一种强大的技术，用于在无标签数据上学习通用语言表示，从而提高NLP任务的性能

1. 预训练阶段: 在大量无标签文本数据上进行预训练，学习通用的语言表示
2. 微调阶段: 针对特定任务进行微调fine-tune，使模型适应各种NLP任务

常见预训练模型

- GPT:基于Transformer的预训练生成模型，学习单向上下文信息
- BERT:基于Transformer的双向编码器表示模型，学习双向上下文信息